📄 Trabajar con CSV en Java

Apuntes completos: leer, escribir, parsear y crear archivos CSV

> *Basado en las clases* `CsvUtils`, `CsvTable`, `CsvPrinter` *y* `LectorCsv`*. Objetivo: entender cada línea, quedarse con las herramientas y poder reutilizarlas en cualquier proyecto.*

📋 Índice

1. ¿Qué es un CSV y por qué importa? (#1-qu%C3%A9-es-un-csv-y-por-qu%C3%A9-importa)
2. Las clases y para qué sirve cada una (#2-las-clases-y-para-qu%C3%A9-sirve-cada-una)
3. Leer un archivo: `BufferedReader` explicado a fondo (#3-leer-un-archivo-bufferedreader-explicado-a-fondo)
4. Parsear una línea CSV: `CsvUtils.parseCSVLine()` diseccionado (#4-parsear-una-l%C3%ADnea-csv-csvutilsparsecsvline-diseccionado)
5. Almacenar los datos: `CsvTable` explicada (#5-almacenar-los-datos-csvtable-explicada)
6. Imprimir la tabla: `CsvPrinter` explicado (#6-imprimir-la-tabla-csvprinter-explicado)
7. Herramienta reutilizable: `CsvUtils` ampliada (#7-herramienta-reutilizable-csvutils-ampliada)
8. Escribir y crear archivos CSV desde cero (#8-escribir-y-crear-archivos-csv-desde-cero)
9. Clase `LectorCsv` refactorizada y reutilizable (#9-clase-lectorcsv-refactorizada-y-reutilizable)
10. Clase `EscritorCsv` reutilizable (#10-clase-escritorcsv-reutilizable)
11. Casos especiales y errores comunes (#11-casos-especiales-y-errores-comunes)
12. Cheatsheet rápida (#12-cheatsheet-r%C3%A1pida)

---

1. ¿Qué es un CSV y por qué importa?

Un **CSV** (Comma-Separated Values) es simplemente un archivo de texto donde los datos están separados por comas. No hay magia. Ábrelo con el Bloc de Notas y verás esto:

CÓDIGO

```
id,nombre,email,saldo
1,Ana García,ana@correo.com,1500.00
2,Luis Pérez,"Pérez, Luis",320.50
3,Marta,"López, ""Marta""",980.00
```

La **primera línea** es siempre la cabecera (los nombres de las columnas).
Las **siguientes líneas** son los datos, una fila por línea.

El problema que hace falta resolver

El separador es la coma. Pero ¿qué pasa si un campo contiene una coma?

```
Ana,"López, García",Madrid   ← el apellido tiene coma dentro
```

La solución estándar CSV es **envolver el campo en comillas dobles**.
Y si el campo además contiene comillas, se escapan duplicándolas:

```
"Empresa ""La Buena"", S.L."  →  resultado: Empresa "La Buena", S.L.
```

Por eso no podemos simplemente usar `linea.split(",")`. Necesitamos un parser que entienda estas reglas. Eso es exactamente lo que hace `CsvUtils.parseCSVLine()`.

---

2. Las clases y para qué sirve cada una

```
┌──────────────────────────────────────────────────────────┐
│                   ARQUITECTURA CSV                        │
│                                                           │
│  DISCO DURO          JAVA (memoria)          PANTALLA     │
│  ────────           ────────────           ───────       │
│                                                           │
│  archivo.csv  →  LectorCsv        →  CsvTable  →  CsvPrinter  │
│                  (abre y lee)        (almacena)   (imprime)    │
│                       │                                   │
│                  CsvUtils                                 │
│                  (parsea líneas)                          │
│                                                           │
│  CsvTable  →  EscritorCsv  →  archivo.csv                 │
│  (datos)      (escribe)       (en disco)                  │
└──────────────────────────────────────────────────────────┘
```

CLASE	RESPONSABILIDAD	ANALOGÍA
`CsvUtils`	Cortar una línea en campos	El cuchillo de cocina
`CsvTable`	Guardar la tabla en memoria	La bandeja donde pones los trozos
`LectorCsv`	Abrir el archivo y leerlo	El que va a buscar los ingredientes
`CsvPrinter`	Mostrar la tabla formateada	El camarero que la presenta
`EscritorCsv`	Escribir datos en CSV	El que guarda las sobras en el tupper

---

3. Leer un archivo: `BufferedReader` explicado a fondo

El problema: leer un archivo en Java

Java no puede leer un archivo directamente como si fuera una variable. Necesita abrir un "canal" hacia el archivo, leer por ese canal y cerrarlo al terminar.

Las clases involucradas

CÓDIGO

```
Path           → dónde está el archivo (la dirección)
Files          → la puerta de entrada a las operaciones de archivo
BufferedReader → el lector eficiente que lee línea a línea
```

Paso a paso desde cero

CÓDIGO

```java
// PASO 1: Decirle a Java dónde está el archivo
// Path.of() construye una ruta de forma portátil (funciona en Windows, Mac y Linux)
// En Windows: Path.of("C:/datos/clientes.csv")
// En Linux:   Path.of("/home/marcos/datos/clientes.csv")
// Relativa:   Path.of("data", "clientes.csv")  ← recomendado para proyectos
Path ruta = Path.of("data", "clientes.csv");

// PASO 2: Verificar si el archivo existe antes de intentar abrirlo
if (!Files.exists(ruta)) {
    System.out.println("El archivo no existe en: " + ruta.toAbsolutePath());
    return;
}

// PASO 3: Abrir el archivo con BufferedReader
// Files.newBufferedReader() abre el archivo y devuelve un lector.
// StandardCharsets.UTF_8 → usa codificación UTF-8 (soporta tildes y ñ).
// El try-with-resources (try con paréntesis) garantiza que el archivo
// se cierre automáticamente aunque haya una excepción. SIEMPRE úsalo.
try (BufferedReader lector = Files.newBufferedReader(ruta, StandardCharsets.UTF_8)) {

    // PASO 4: Leer la primera línea (cabeceras)
    String linea = lector.readLine();
    // readLine() devuelve null cuando llega al final del archivo.
    // Si la primera línea es null, el archivo está vacío.

    if (linea == null) {
        System.out.println("El archivo está vacío.");
        return;
    }

    System.out.println("Cabeceras: " + linea);

    // PASO 5: Leer el resto de líneas con un bucle while
    // El truco del while: leer y asignar dentro de la condición.
    // Cuando readLine() devuelva null (fin de archivo), el bucle para.
    while ((linea = lector.readLine()) != null) {
        System.out.println("Línea de datos: " + linea);
    }

} catch (IOException e) {
    // IOException cubre: archivo no encontrado, sin permisos, disco lleno...
    System.err.println("Error leyendo el archivo: " + e.getMessage());
}
```

¿Por qué `BufferedReader` y no `FileReader` directamente?

`FileReader` lee carácter a carácter. Cada lectura es una operación de disco. Para un archivo de 10.000 líneas serían 500.000 operaciones de disco. Lento.

`BufferedReader` carga un bloque grande de texto en memoria (el "buffer") y de ahí lee. Mucho más rápido. Además añade el método `readLine()` que `FileReader` no tiene.

CÓDIGO

```
Sin buffer:  [disco] → ch → [memoria]   (una operación por carácter)
Con buffer:  [disco] → [▓▓▓▓▓▓▓ buffer ▓▓▓▓▓▓▓] → ch → [memoria]  (una operación por bloque)
```

¿Qué es `try-with-resources`?

Es la forma moderna de gestionar recursos (archivos, conexiones, etc.) en Java.

CÓDIGO

```
// SIN try-with-resources (código antiguo y peligroso)
BufferedReader lector = null;
try {
    lector = Files.newBufferedReader(ruta, StandardCharsets.UTF_8);
    // ... leer ...
} catch (IOException e) {
    e.printStackTrace();
} finally {
    if (lector != null) {
        try { lector.close(); } catch (IOException e) { /* ignorar */ }
    }
}

// CON try-with-resources (Java 7+, limpio y seguro)
try (BufferedReader lector = Files.newBufferedReader(ruta, StandardCharsets.UTF_8)) {
    // ... leer ...
    // lector.close() se llama automáticamente al salir del bloque,
    // tanto si termina bien como si lanza una excepción.
}
```

---

4. Parsear una línea CSV: `CsvUtils.parseCSVLine()` diseccionado

El algoritmo explicado con trazas

Tomemos esta línea de ejemplo y sigamos el algoritmo carácter por carácter:

CÓDIGO

```
Ana,"López, García",28
```

El algoritmo usa tres variables:

- `out` → la lista donde vamos guardando los campos terminados
- `sb` → un "cuaderno de borrador" donde construimos el campo actual
- `enComillas` → un interruptor booleano: ¿estamos dentro de `"..."` ?

CÓDIGO

```
Carácter  enComillas  Acción                       sb             out
──────────────────────────────────────────────────────────────────
A         false       carácter normal → añadir       "A"            []
n         false       carácter normal → añadir       "An"           []
a         false       carácter normal → añadir       "Ana"          []
,         false       COMA + NO comillas → fin campo ""             ["Ana"]
"         false       comilla → activar interruptor  ""             ["Ana"]
L         true        carácter normal → añadir       "L"            ["Ana"]
ó         true        carácter normal → añadir       "Ló"           ["Ana"]
p         true        carácter normal → añadir       "Lóp"          ["Ana"]
e         true        carácter normal → añadir       "Lópe"         ["Ana"]
z         true        carácter normal → añadir       "López"        ["Ana"]
,         true        COMA pero SÍ comillas → añadir "López,"       ["Ana"]
(espacio) true        carácter normal → añadir       "López, "      ["Ana"]
G         true        carácter normal → añadir       "López, G"     ["Ana"]
a         true        carácter normal → añadir       "López, Ga"    ["Ana"]
r         true        carácter normal → añadir       "López, Gar"   ["Ana"]
c         true        carácter normal → añadir       "López, Garc"  ["Ana"]
í         true        carácter normal → añadir       "López, Garcí" ["Ana"]
a         true        carácter normal → añadir       "López, García" ["Ana"]
"         true        comilla → desactivar interruptor "López, García" ["Ana"]
,         false       COMA + NO comillas → fin campo ""             ["Ana","López, García"]
2         false       carácter normal → añadir       "2"            ["Ana","López, García"]
8         false       carácter normal → añadir       "28"           ["Ana","López, García"]
──────────────────────────────────────────────────────────────────
FIN BUCLE             añadir último campo            ""             ["Ana","López, García","28"]
```

Resultado: `["Ana", "López, García", "28"]` ✅

El código con comentarios extendidos

CÓDIGO

```
public static List<String> parseCSVLine(String linea) {

    List<String> campos = new ArrayList<>();
    StringBuilder campoActual = new StringBuilder();
    boolean dentroDeComillas = false;

    for (int i = 0; i < linea.length(); i++) {
        char c = linea.charAt(i);

        // ── CASO 1: El carácter es una comilla doble ──────────────
        if (c == '"') {

            // ¿Estamos dentro de comillas Y el siguiente carácter TAMBIÉN es comilla?
            // Eso es una comilla escapada: "" → significa una " literal dentro del campo.
            // Ejemplo: "Di ""hola""" → resultado: Di "hola"
            if (dentroDeComillas && i + 1 < linea.length() && linea.charAt(i + 1) == '"') {
                campoActual.append('"');  // Añadimos UNA comilla al campo
                i++;                      // Saltamos la segunda comilla del par
            }
            // Si no es comilla escapada, es el inicio o fin de un campo entrecomillado.
            // Simplemente alternamos el estado del interruptor.
            else {
                dentroDeComillas = !dentroDeComillas;
                // OJO: la comilla envolvente NO se añade al campo.
                // "Ana" → el campo será Ana, sin las comillas.
            }
        }

        // ── CASO 2: El carácter es una coma ───────────────────────
        else if (c == ',' && !dentroDeComillas) {
            // Coma FUERA de comillas = separador de campo.
            // El campo actual está completo: lo añadimos a la lista.
            campos.add(campoActual.toString());
            // Reiniciamos el acumulador para el siguiente campo.
            // setLength(0) es más eficiente que crear un new StringBuilder().
            campoActual.setLength(0);
        }

        // ── CASO 3: Cualquier otro carácter ───────────────────────
        else {
            // Letra, número, espacio, símbolo... lo añadimos al campo actual.
            // También entra aquí la coma si dentroDeComillas == true.
            campoActual.append(c);
        }
    }

    // Después del bucle, el último campo no habrá sido añadido
    // porque no termina en coma. Lo añadimos manualmente.
    campos.add(campoActual.toString());

    return campos;
}
```

¿Por qué `StringBuilder` y no concatenar Strings?

CÓDIGO

```
// MAL: cada += crea un nuevo objeto String en memoria
String campo = "";
campo += 'A';   // crea "A"
campo += 'n';   // crea "An", descarta "A"
campo += 'a';   // crea "Ana", descarta "An"
// Para una línea con 100 caracteres: 100 objetos String creados y descartados

// BIEN: StringBuilder modifica el mismo objeto interno
StringBuilder sb = new StringBuilder();
sb.append('A');  // modifica el buffer interno
sb.append('n');  // modifica el buffer interno
sb.append('a');  // modifica el buffer interno
sb.toString();   // SOLO aquí crea el String final: "Ana"
// Para una línea con 100 caracteres: 1 objeto String creado
```

---

5. Almacenar los datos: `CsvTable` explicada

`CsvTable` es un contenedor genérico en memoria. No sabe qué tipo de CSV vas a leer. Solo sabe que tiene cabeceras y filas.

Estructura interna

CÓDIGO

```
CsvTable
├── headers: List<String>
|   └── ["id", "nombre", "email", "saldo"]
|
└── rows: List<Map<String, String>>
    ├── fila 0: {"id"="1", "nombre"="Ana", "email"="ana@c.com", "saldo"="1500"}
    ├── fila 1: {"id"="2", "nombre"="Luis", "email"="luis@c.com", "saldo"="320"}
    └── fila 2: {"id"="3", "nombre"="Marta", "email"="marta@c.com", "saldo"="980"}
```

¿Por qué `Map<String, String>` para las filas?

Porque así accedes a los campos por nombre, no por posición:

CÓDIGO

```
// SIN Map (por posición) → frágil, si el CSV cambia el orden, todo se rompe
String nombre = campos.get(1);   // ¿Qué hay en la posición 1? Hay que contarlo.

// CON Map (por nombre) → robusto, claro y autoexplicativo
String nombre = fila.get("nombre");  // Está claro qué estás cogiendo.
```

El código con comentarios extendidos

CÓDIGO

```java
public class CsvTable {

    // 'final' en la referencia: el objeto List nunca cambia,
    // aunque su contenido (los elementos dentro) sí puede modificarse.
    private final List<String> headers = new ArrayList<>();
    private final List<Map<String, String>> rows = new ArrayList<>();

    // ── GETTERS con vista inmodificable ──────────────────────
    // Collections.unmodifiableList() devuelve una "envoltura" de solo lectura.
    // Quien llame a getHeaders() no podrá hacer headers.add("nueva") desde fuera.
    // Protege los datos internos sin copiarlos (eficiente).
    public List<String> getHeaders() {
        return Collections.unmodifiableList(headers);
    }

    public List<Map<String, String>> getRows() {
        return Collections.unmodifiableList(rows);
    }

    // ── SETTERS de cabeceras ──────────────────────────────────
    public void setHeaders(List<String> hdrs) {
        headers.clear();      // Borra las anteriores (si las hay)
        headers.addAll(hdrs); // Copia todas las nuevas
    }

    // ── AÑADIR FILA ───────────────────────────────────────────
    // Recibe una lista de valores en el mismo orden que las cabeceras
    // y construye el Map que asocia cada cabecera con su valor.
    public void addRow(List<String> valores) {

        // LinkedHashMap mantiene el orden de inserción.
        // Si usáramos HashMap, el orden de las columnas sería aleatorio.
        Map<String, String> fila = new LinkedHashMap<>();

        for (int i = 0; i < headers.size(); i++) {
            String clave = headers.get(i);
            // Si hay menos valores que cabeceras, el campo faltante es "".
            // Esto evita IndexOutOfBoundsException con CSVs mal formados.
            String valor = (i < valores.size()) ? valores.get(i) : "";
            fila.put(clave, valor);
        }

        rows.add(fila);
    }

    public int size() { return rows.size(); }
}
```

Acceder a los datos de la tabla

CÓDIGO

```
CsvTable tabla = LectorCsv.leer(Path.of("data/clientes.csv"));

// Leer las cabeceras
List<String> cabeceras = tabla.getHeaders();
System.out.println(cabeceras); // [id, nombre, email, saldo]

// Leer todas las filas
for (Map<String, String> fila : tabla.getRows()) {
    String nombre = fila.get("nombre");
    String saldo  = fila.get("saldo");
    System.out.println(nombre + " tiene " + saldo + "€");
}

// Acceder a una fila concreta por índice
Map<String, String> primeraFila = tabla.getRows().get(0);
System.out.println(primeraFila.get("nombre")); // Ana

// Número de filas (sin contar la cabecera)
System.out.println("Total registros: " + tabla.size());
```

---

6. Imprimir la tabla: `CsvPrinter` explicado

`CsvPrinter` resuelve el problema de imprimir columnas alineadas. Sin él:

CÓDIGO

```
id nombre email
1 Ana ana@correo.com
2 Luis luis@correo.es
```

Con él:

CÓDIGO

```
id    nombre    email
──────────────────────
1     Ana       ana@correo.com
2     Luis      luis@correo.es
```

El truco del ancho máximo

El algoritmo calcula, para cada columna, cuál es el texto más largo entre la cabecera y todos los valores. Ese será el ancho de la columna.

```java
// Paso 1: el ancho mínimo de cada columna es el largo de su cabecera
int[] anchos = new int[cabeceras.size()];
for (int i = 0; i < cabeceras.size(); i++) {
    anchos[i] = cabeceras.get(i).length();
}

// Paso 2: recorrer todos los datos y actualizar si algún valor es más largo
for (Map<String, String> fila : tabla.getRows()) {
    for (int i = 0; i < cabeceras.size(); i++) {
        String valor = fila.get(cabeceras.get(i));
        if (valor != null) {
            // Math.max devuelve el mayor de los dos números
            anchos[i] = Math.max(anchos[i], valor.length());
        }
    }
}
```

El método `rellenarPavo` (relleno con espacios)

```java
// Rellena un texto con espacios a la derecha hasta alcanzar el ancho dado.
// "Ana" con ancho 10 → "Ana       "
private static String rellenarPavo(String texto, int ancho) {
    if (texto == null) texto = ""; // protección contra null
    if (texto.length() >= ancho) return texto; // ya es suficientemente largo
    return texto + " ".repeat(ancho - texto.length());
    // " ".repeat(n) crea una cadena de n espacios. Disponible desde Java 11.
}
```

> **Nota:** *el nombre* `rellenarPavo` *es un nombre informal de tu profe. En producción se llamaría* `padRight` *o* `rellenarDerecha`*. El concepto es estándar: se llama right-padding.*

---

7. Herramienta reutilizable: `CsvUtils` ampliada

La clase original solo tiene `parseCSVLine`. Vamos a ampliarla con todos los métodos que necesitaremos en el proyecto. Esta es la versión completa y reutilizable:

CÓDIGO

```java
package k82studio.util;

import java.util.ArrayList;
import java.util.List;

/**
 * Utilidades estáticas para trabajar con el formato CSV.
 *
 * Clase de utilidad: constructor privado, todos los métodos son estáticos.
 * No se instancia. Se usa directamente: CsvUtils.parseCSVLine(linea)
 */
public class CsvUtils {

    private CsvUtils() { }

    // ================================================================
    // LEER: parsear una línea CSV a lista de campos
    // ================================================================

    /**
     * Convierte una línea CSV en una lista de campos individuales.
     *
     * Maneja correctamente:
     *   - Separador por coma
     *   - Campos entre comillas: "campo con, coma"
     *   - Comillas escapadas dentro de campos: "tiene ""comillas"" dentro"
     *
     * Ejemplo:
     *   Entrada:  Ana,"López, García","Di ""hola""",28
     *   Salida:   ["Ana", "López, García", "Di \"hola\"", "28"]
     *
     * @param linea La línea CSV completa
     * @return Lista de campos (nunca null, puede estar vacía)
     */
    public static List<String> parseCSVLine(String linea) {
        List<String> campos = new ArrayList<>();

        if (linea == null || linea.isBlank()) {
            return campos; // línea vacía → lista vacía
        }

        StringBuilder campoActual = new StringBuilder();
        boolean dentroDeComillas = false;

        for (int i = 0; i < linea.length(); i++) {
            char c = linea.charAt(i);

            if (c == '"') {
                // Comilla escapada: "" dentro de un campo entre comillas
                if (dentroDeComillas && i + 1 < linea.length() && linea.charAt(i + 1) == '"') {
                    campoActual.append('"');
                    i++;
                } else {
                    dentroDeComillas = !dentroDeComillas;
                }
            } else if (c == ',' && !dentroDeComillas) {
                campos.add(campoActual.toString());
                campoActual.setLength(0);
            } else {
                campoActual.append(c);
            }
        }

        campos.add(campoActual.toString()); // último campo
        return campos;
    }
```

```java
// ================================================================
// ESCRIBIR: convertir un campo a formato CSV seguro
// ================================================================

/**
 * Convierte un valor al formato CSV correcto para escribir en archivo.
 *
 * Reglas:
 *   - Si el valor es null → devuelve ""
 *   - Si contiene coma, comilla o salto de línea → lo envuelve en comillas
 *     y duplica las comillas internas
 *   - Si no tiene caracteres especiales → lo devuelve tal cual
 *
 * Ejemplos:
 *   "Ana"           → Ana
 *   "López, García" → "López, García"
 *   "Di \"hola\""   → "Di ""hola"""
 *   null            → (cadena vacía)
 *
 * @param valor El texto del campo
 * @return El campo listo para insertar en una línea CSV
 */
public static String escaparCampo(String valor) {
    if (valor == null) return "";

    // ¿Contiene algún carácter que requiera entrecomillado?
    boolean necesitaComillas = valor.contains(",")
                             || valor.contains("\"")
                             || valor.contains("\n")
                             || valor.contains("\r");

    if (necesitaComillas) {
        // Duplicar las comillas internas y envolver en comillas
        return "\"" + valor.replace("\"", "\"\"") + "\"";
    }

    return valor;
}

/**
 * Convierte una lista de valores en una línea CSV lista para escribir.
 *
 * Ejemplo:
 *   ["Ana", "López, García", "28"] → Ana,"López, García",28
 *
 * @param valores Lista de campos en orden
 * @return Línea CSV completa (sin salto de línea al final)
 */
public static String construirLineaCsv(List<String> valores) {
    StringBuilder sb = new StringBuilder();

    for (int i = 0; i < valores.size(); i++) {
        sb.append(escaparCampo(valores.get(i)));
        if (i < valores.size() - 1) {
            sb.append(',');
        }
    }

    return sb.toString();
}

/**
 * Versión varargs: acepta campos directamente sin crear una lista.
 *
 * Ejemplo:
 *   CsvUtils.construirLineaCsv("1", "Ana", "ana@correo.com", "1500.0")
```

```java
     *   → 1,Ana,ana@correo.com,1500.0
     *
     * @param campos Los campos en orden (número variable de argumentos)
     * @return Línea CSV completa
     */
    public static String construirLineaCsv(String... campos) {
        return construirLineaCsv(List.of(campos));
    }

    // ================================================================
    // UTILIDADES DE CONVERSIÓN SEGURA
    // ================================================================

    /**
     * Convierte un campo CSV (String) a double de forma segura.
     * Si el campo está vacío o no es un número válido, devuelve el valor por defecto.
     *
     * Acepta tanto punto como coma como separador decimal.
     *
     * Ejemplos:
     *   "45.50"  → 45.5
     *   "45,50"  → 45.5  (coma reemplazada por punto)
     *   ""       → 0.0   (valor por defecto)
     *   "abc"    → 0.0   (no es número, devuelve por defecto)
     *
     * @param campo          El texto del campo CSV
     * @param valorPorDefecto Valor a devolver si no se puede convertir
     * @return El double resultante
     */
    public static double parseDouble(String campo, double valorPorDefecto) {
        if (campo == null || campo.isBlank()) return valorPorDefecto;
        try {
            return Double.parseDouble(campo.trim().replace(",", "."));
        } catch (NumberFormatException e) {
            return valorPorDefecto;
        }
    }

    /**
     * Convierte un campo CSV a int de forma segura.
     *
     * @param campo          El texto del campo
     * @param valorPorDefecto Valor si no se puede convertir
     * @return El int resultante
     */
    public static int parseInt(String campo, int valorPorDefecto) {
        if (campo == null || campo.isBlank()) return valorPorDefecto;
        try {
            return Integer.parseInt(campo.trim());
        } catch (NumberFormatException e) {
            return valorPorDefecto;
        }
    }

    /**
     * Limpia un campo CSV: elimina espacios al inicio y al final.
     * Nunca devuelve null: si el campo es null devuelve "".
     *
     * @param campo El texto del campo
     * @return El campo limpio
     */
    public static String limpiar(String campo) {
        if (campo == null) return "";
        return campo.trim();
    }
}
```

8. Escribir y crear archivos CSV desde cero

Leer es la mitad del trabajo. Aquí aprendemos a **escribir**.

Conceptos clave para escribir archivos

CÓDIGO

```
// Files.writeString()  → escribe un String completo de una vez (para archivos pequeños)
// BufferedWriter       → escribe línea a línea de forma eficiente (recomendado)
// StandardOpenOption   → controla si creas, sobreescribes o añades al final
```

Las opciones de escritura

CÓDIGO

```
// OPCIÓN 1: Crear o sobreescribir (comportamiento por defecto)
// Si el archivo existe → lo borra y lo crea de nuevo.
// Si no existe         → lo crea.
try (BufferedWriter escritor = Files.newBufferedWriter(ruta, StandardCharsets.UTF_8)) {
    escritor.write("contenido");
}

// OPCIÓN 2: Añadir al final (APPEND) - no borra el contenido existente
try (BufferedWriter escritor = Files.newBufferedWriter(
        ruta,
        StandardCharsets.UTF_8,
        StandardOpenOption.CREATE,   // crear si no existe
        StandardOpenOption.APPEND    // añadir al final si existe
)) {
    escritor.write("nueva línea al final");
}

// OPCIÓN 3: Solo crear (falla si ya existe, protege de sobreescrituras accidentales)
try (BufferedWriter escritor = Files.newBufferedWriter(
        ruta,
        StandardCharsets.UTF_8,
        StandardOpenOption.CREATE_NEW  // falla con FileAlreadyExistsException si existe
)) {
    escritor.write("archivo nuevo");
}
```

Crear un CSV desde cero, paso a paso

CÓDIGO

```
Path ruta = Path.of("data", "salida.csv");

// Crear el directorio si no existe
Files.createDirectories(ruta.getParent());

try (BufferedWriter escritor = Files.newBufferedWriter(ruta, StandardCharsets.UTF_8)) {

    // 1. Escribir la cabecera
    escritor.write(CsvUtils.construirLineaCsv("id", "nombre", "email", "saldo"));
    escritor.newLine(); // salto de línea compatible con el SO actual

    // 2. Escribir filas de datos
    escritor.write(CsvUtils.construirLineaCsv("1", "Ana García", "ana@correo.com", "1500.0"));
    escritor.newLine();

    escritor.write(CsvUtils.construirLineaCsv("2", "Luis, el jefe", "luis@correo.com", "320.5"));
    escritor.newLine();
    // "Luis, el jefe" contiene coma → CsvUtils.escaparCampo lo entrecomillará automáticamente

} catch (IOException e) {
    System.err.println("Error escribiendo el CSV: " + e.getMessage());
}
```

Resultado en disco (`salida.csv`)

CÓDIGO

```
id,nombre,email,saldo
1,Ana García,ana@correo.com,1500.0
2,"Luis, el jefe",luis@correo.com,320.5
```

---

9. Clase `LectorCsv` refactorizada y reutilizable

La versión original del ejercicio tenía el `main` dentro y la ruta hardcodeada. Esta es la versión reutilizable, lista para usar en cualquier proyecto:

CÓDIGO

```java
package k82studio.util;

import java.io.BufferedReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.NoSuchFileException;
import java.nio.file.Path;
import java.util.List;

/**
 * Utilidad para leer archivos CSV y cargarlos en una CsvTable.
 *
 * Uso básico:
 *   CsvTable tabla = LectorCsv.leer(Path.of("data/clientes.csv"));
 *
 * Uso con manejo de errores detallado:
 *   try {
 *       CsvTable tabla = LectorCsv.leer(ruta);
 *   } catch (NoSuchFileException e) {
 *       System.out.println("Archivo no encontrado");
 *   } catch (IOException e) {
 *       System.out.println("Error de lectura");
 *   }
 */
public class LectorCsv {

    // Constructor privado: solo métodos estáticos, no se instancia.
    private LectorCsv() { }

    // ================================================================
    // MÉTODO PRINCIPAL: leer CSV → CsvTable
    // ================================================================

    /**
     * Lee un archivo CSV y devuelve su contenido como CsvTable.
     *
     * La primera línea del archivo se trata como cabecera.
     * Las siguientes líneas como filas de datos.
     * Las líneas vacías se ignoran.
     * Las líneas con formato incorrecto se saltan con aviso en consola.
     *
     * @param ruta Ruta al archivo CSV
     * @return CsvTable con los datos cargados (vacía si el archivo está vacío)
     * @throws NoSuchFileException Si el archivo no existe
     * @throws IOException         Si ocurre cualquier otro error de lectura
     */
    public static CsvTable leer(Path ruta) throws IOException {

        CsvTable tabla = new CsvTable();

        // Verificación explícita: mejor mensaje de error que el genérico
        if (!Files.exists(ruta)) {
            throw new NoSuchFileException(
                "Archivo no encontrado: " + ruta.toAbsolutePath()
            );
        }

        try (BufferedReader lector = Files.newBufferedReader(ruta, StandardCharsets.UTF_8)) {

            // ── Leer cabeceras ──────────────────────────────
            String linea = lector.readLine();

            if (linea == null || linea.isBlank()) {
                System.out.println("⚠️ CSV vacío o sin cabeceras: " + ruta.getFileName());
                return tabla; // Tabla vacía
```

```java
            }

            List<String> cabeceras = CsvUtils.parseCSVLine(linea);
            tabla.setHeaders(cabeceras);

            // ── Leer filas de datos ──────────────────────────
            int numeroLinea = 1;

            while ((linea = lector.readLine()) != null) {
                numeroLinea++;

                // Saltar líneas en blanco
                if (linea.isBlank()) continue;

                // Parsear y añadir la fila
                try {
                    List<String> campos = CsvUtils.parseCSVLine(linea);
                    tabla.addRow(campos);
                } catch (Exception e) {
                    // Línea corrupta: avisar pero continuar leyendo las demás
                    System.err.println("⚠️ Línea " + numeroLinea + " ignorada: " + e.getMessage());
                }
            }
        }

        System.out.println("✅ Leídas " + tabla.size() + " filas de: " + ruta.getFileName());
        return tabla;
    }

    // ═══════════════════════════════════════════════════════════════
    // MÉTODO ALTERNATIVO: leer con valor por defecto si no existe
    // ═══════════════════════════════════════════════════════════════

    /**
     * Igual que leer(), pero si el archivo no existe devuelve una tabla vacía
     * en lugar de lanzar excepción.
     *
     * Útil en el arranque de la app: si es la primera vez que se ejecuta,
     * no hay CSV todavía y está bien que la tabla esté vacía.
     *
     * @param ruta Ruta al archivo CSV
     * @return CsvTable con datos (o vacía si el archivo no existe)
     */
    public static CsvTable leerOVacio(Path ruta) {
        try {
            return leer(ruta);
        } catch (NoSuchFileException e) {
            System.out.println("ℹ️ Archivo no existe aún, empezando vacío: " + ruta.getFileName());
            return new CsvTable();
        } catch (IOException e) {
            System.err.println("❌ Error leyendo " + ruta.getFileName() + ": " + e.getMessage());
            return new CsvTable();
        }
    }
}
```

---

10. Clase `EscritorCsv` reutilizable

La contrapartida del lector. Escribe una `CsvTable` en disco:

CÓDIGO

```java
package k82studio.util;

import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.Map;

/**
 * Utilidad para escribir CsvTable en archivos CSV.
 *
 * Uso básico:
 *   EscritorCsv.escribir(tabla, Path.of("data/salida.csv"));
 *
 * Añadir una línea al final sin reescribir todo:
 *   EscritorCsv.añadirFila(campos, Path.of("data/salida.csv"));
 */
public class EscritorCsv {

    private EscritorCsv() { }

    // ================================================================
    // ESCRIBIR TABLA COMPLETA (sobreescribe el archivo)
    // ================================================================

    /**
     * Escribe una CsvTable completa en un archivo CSV.
     * Si el archivo existe lo sobreescribe. Si no existe lo crea.
     * Crea los directorios intermedios necesarios automáticamente.
     *
     * @param tabla La tabla con cabeceras y filas a escribir
     * @param ruta  Ruta del archivo de destino
     * @throws IOException Si no se puede crear o escribir el archivo
     */
    public static void escribir(CsvTable tabla, Path ruta) throws IOException {

        // Crear directorios intermedios si no existen
        // createDirectories no falla si el directorio ya existe
        Files.createDirectories(ruta.getParent());

        try (BufferedWriter escritor = Files.newBufferedWriter(ruta, StandardCharsets.UTF_8)) {

            // ── Escribir cabecera ──────────────────────────
            List<String> cabeceras = tabla.getHeaders();
            escritor.write(CsvUtils.construirLineaCsv(cabeceras));
            escritor.newLine();

            // ── Escribir cada fila ─────────────────────────
            for (Map<String, String> fila : tabla.getRows()) {

                // Reconstruir la lista de valores en el mismo orden que las cabeceras
                List<String> valores = cabeceras.stream()
                        .map(cab -> fila.getOrDefault(cab, ""))
                        .toList(); // Java 16+, o .collect(Collectors.toList()) en versiones anteriores

                escritor.write(CsvUtils.construirLineaCsv(valores));
                escritor.newLine();
            }
        }

        System.out.println("✅ CSV guardado: " + ruta.toAbsolutePath()
                + " (" + tabla.size() + " filas)");
    }
```

```java
    // ================================================================
    // CREAR ARCHIVO VACÍO CON CABECERAS
    // ================================================================

    /**
     * Crea un CSV nuevo con solo la cabecera y sin filas.
     * Útil para inicializar el archivo en el primer arranque de la app.
     * Si el archivo ya existe, NO lo sobreescribe (operación segura).
     *
     * @param cabeceras Las columnas del CSV
     * @param ruta      Ruta del archivo a crear
     * @throws IOException Si no se puede crear
     */
    public static void crearVacio(List<String> cabeceras, Path ruta) throws IOException {

        if (Files.exists(ruta)) {
            System.out.println("ℹ️ El archivo ya existe, no se sobreescribe: " + ruta.getFileName());
            return;
        }

        Files.createDirectories(ruta.getParent());

        try (BufferedWriter escritor = Files.newBufferedWriter(ruta, StandardCharsets.UTF_8)) {
            escritor.write(CsvUtils.construirLineaCsv(cabeceras));
            escritor.newLine();
        }

        System.out.println("✅ CSV creado vacío: " + ruta.toAbsolutePath());
    }

    // ================================================================
    // AÑADIR UNA FILA AL FINAL (SIN REESCRIBIR TODO)
    // ================================================================

    /**
     * Añade una sola fila al final del CSV sin reescribir el archivo completo.
     * Más eficiente cuando solo hay que añadir datos nuevos.
     *
     * PRECAUCIÓN: no verifica que el número de campos coincida con la cabecera.
     * Úsalo cuando estés seguro del formato.
     *
     * @param campos Lista de valores de la nueva fila (en orden de columnas)
     * @param ruta   Ruta del archivo CSV existente
     * @throws IOException Si no se puede escribir
     */
    public static void añadirFila(List<String> campos, Path ruta) throws IOException {

        // APPEND: abre el archivo y escribe al final sin borrar lo que hay
        try (BufferedWriter escritor = Files.newBufferedWriter(
                ruta,
                StandardCharsets.UTF_8,
                java.nio.file.StandardOpenOption.CREATE,
                java.nio.file.StandardOpenOption.APPEND
        )) {
            escritor.write(CsvUtils.construirLineaCsv(campos));
            escritor.newLine();
        }
    }
}
```

11. Casos especiales y errores comunes

❌ Error 1: usar `split(",")` directamente

CÓDIGO

```
// MAL: se rompe con campos que contienen comas
String[] campos = linea.split(",");
// "Ana","López, García","28" → ["Ana", "\"López", " García\"", "\"28\""]

// BIEN: usar el parser
List<String> campos = CsvUtils.parseCSVLine(linea);
```

❌ Error 2: olvidar el `newLine()` al escribir

CÓDIGO

```
// MAL: todas las filas quedan en la misma línea
escritor.write("1,Ana");
escritor.write("2,Luis");
// Resultado: 1,Ana2,Luis

// BIEN
escritor.write("1,Ana");
escritor.newLine(); // ← imprescindible
escritor.write("2,Luis");
escritor.newLine();
```

❌ Error 3: no cerrar el archivo

CÓDIGO

```
// MAL: el archivo puede quedar bloqueado o los datos sin escribir
BufferedWriter escritor = Files.newBufferedWriter(ruta, StandardCharsets.UTF_8);
escritor.write("datos");
// Si hay una excepción aquí, el archivo nunca se cierra

// BIEN: try-with-resources cierra automáticamente
try (BufferedWriter escritor = Files.newBufferedWriter(ruta, StandardCharsets.UTF_8)) {
    escritor.write("datos");
} // ← se cierra aquí siempre
```

❌ Error 4: hardcodear rutas absolutas

CÓDIGO

```
// MAL: solo funciona en tu máquina
Path ruta = Path.of("F:/Marcos/Programacion/Java/data/clientes.csv");

// BIEN: ruta relativa al directorio de trabajo del proyecto
Path ruta = Path.of("data", "clientes.csv");
// Equivale a: [directorio-del-proyecto]/data/clientes.csv
```

❌ Error 5: olvidar manejar líneas vacías

CÓDIGO

```
// MAL: falla con IndexOutOfBoundsException en líneas vacías
while ((linea = lector.readLine()) != null) {
    List<String> campos = CsvUtils.parseCSVLine(linea); // ← falla si linea=""
    tabla.addRow(campos);
}

// BIEN
while ((linea = lector.readLine()) != null) {
    if (linea.isBlank()) continue; // saltar líneas vacías
    List<String> campos = CsvUtils.parseCSVLine(linea);
    tabla.addRow(campos);
}
```

❌ Error 6: usar coma como separador decimal en el CSV

CÓDIGO

```
// MAL: "45,50" se interpretará como dos campos: "45" y "50"
escritor.write("1,Ana,45,50"); // ← confusión: ¿son 4 campos o 3?

// BIEN: usar punto como separador decimal en el CSV
escritor.write("1,Ana,45.50");

// Si el usuario introduce "45,50", convertirlo antes de guardar:
String importeLimpio = importe.replace(",", ".");
```

---

12. Cheatsheet rápida

Leer un CSV

CÓDIGO

```
// Opción A: lanza excepción si no existe (recomendado en producción)
CsvTable tabla = LectorCsv.leer(Path.of("data/archivo.csv"));

// Opción B: devuelve tabla vacía si no existe (útil en primer arranque)
CsvTable tabla = LectorCsv.leerOVacio(Path.of("data/archivo.csv"));
```

Recorrer los datos

CÓDIGO

```
// Acceder a las cabeceras
List<String> cols = tabla.getHeaders(); // ["id", "nombre", "saldo"]

// Recorrer todas las filas
for (Map<String, String> fila : tabla.getRows()) {
    String nombre = fila.get("nombre");
    double saldo  = CsvUtils.parseDouble(fila.get("saldo"), 0.0);
}

// Número de filas
int total = tabla.size();
```

Parsear una línea suelta

CÓDIGO

```
List<String> campos = CsvUtils.parseCSVLine("Ana,\"López, García\",28");
// → ["Ana", "López, García", "28"]
```

Construir una línea CSV

CÓDIGO

```
// Con varargs (cómodo para pocas columnas)
String linea = CsvUtils.construirLineaCsv("1", "Ana", "ana@c.com", "1500.0");
// → 1,Ana,ana@c.com,1500.0

// Con lista (cómodo cuando tienes los datos en una lista)
String linea = CsvUtils.construirLineaCsv(List.of("1", "López, García", "1500.0"));
// → 1,"López, García",1500.0
```

Escapar un campo individualmente

CÓDIGO

```
CsvUtils.escaparCampo("Ana")              // → Ana
CsvUtils.escaparCampo("López, García")    // → "López, García"
CsvUtils.escaparCampo("Di \"hola\"")      // → "Di ""hola"""
CsvUtils.escaparCampo(null)               // → (cadena vacía)
```

Escribir un CSV completo

CÓDIGO

```
EscritorCsv.escribir(tabla, Path.of("data/salida.csv"));
```

Crear un CSV vacío con cabeceras

CÓDIGO

```
EscritorCsv.crearVacio(
    List.of("id", "tipo", "importe", "categoria", "descripcion", "fecha"),
    Path.of("data/transacciones.csv")
);
```

Añadir una fila al final (sin reescribir)

CÓDIGO

```
EscritorCsv.añadirFila(
    List.of("abc123", "GASTO", "45.5", "ALIMENTACION", "Mercadona", "2025-03-15"),
    Path.of("data/transacciones.csv")
);
```

Conversiones seguras de campos

CÓDIGO

```
double saldo  = CsvUtils.parseDouble(fila.get("saldo"), 0.0);  // 0.0 si falla
int    edad   = CsvUtils.parseInt(fila.get("edad"), -1);        // -1 si falla
String nombre = CsvUtils.limpiar(fila.get("nombre"));           // trim + null-safe
```

Apuntes creados para el proyecto FinanceApp — k82studio

Fecha: 2025 | Java 17+